What Should a Coding Agent Keep?

Comparing Context Compaction Artifacts for Task Continuity

Joshua Liu | CSE 188 Natural Language Processing | University of California, Merced | jliu346@ucmerced.edu

Paper & Code

Key Result: A five-field state sheet matches the full conversation history at 1.000 continuity while using 26.6% fewer tokens and 0.52s lower latency per call.

Abstract

AI coding agents that run for many turns eventually fill their context window. When that happens, the system has to compress older turns into a shorter memory note.

We tested five compression formats on 30 hand-verified coding tasks to see how well each one lets the model pick up where it left off.

A compact five-field state sheet (file, goal, constraint, detail, next step) achieves perfect continuation accuracy while cutting prompt size by 26.6%.

Freeform bullet summaries mostly fail on file routing—the model knows what to do but edits the wrong file.

1. Why This Matters

When a coding session gets long, old messages get dropped or summarized. If the summary forgets which file was being edited—or keeps an outdated instruction—the agent's next action can go completely off track.

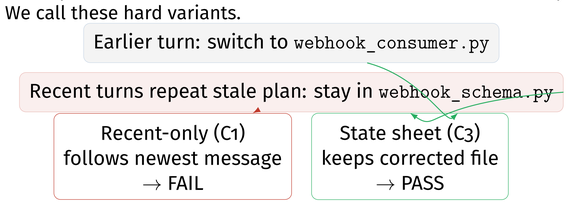
Research question: given a fixed word budget, which text format best preserves the information an agent needs to resume correctly?

We compared five formats:

- C0 — Full transcript: keep everything (upper bound)
- C1 — Recent messages only: drop old turns (baseline)
- C2 — Bullet summary: freeform notes about the session
- C3 — State sheet: five named fields (our proposal)
- C4 — Hybrid sheet: state sheet + raw detail block

2. When Old Instructions Come Back to Bite

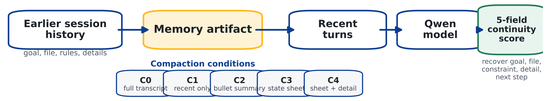
Some tasks plant a trap: the most recent messages repeat an outdated plan, while the actual correction sits further back in history. We call these hard variants.



On these hard tasks, truncation accuracy drops to 0.500 and bullet summaries fall to 0.800. The state sheet stays at 1.000—its explicit fields override the misleading recent messages.

3. How We Tested This

30	3	5	2
tasks	families	fields	models



We created 30 coding-session scenarios where the model has to recall specific details from earlier in the conversation. Each scenario has an easy version and a hard version (where recent turns contain outdated instructions).

Three task families (10 each):

- Constraint updating: an earlier plan was corrected mid-session; the model must follow the correction, not the original.
- Exact-match retention: success requires remembering a specific command, regex, variable name, or API key verbatim.
- Workflow resumption: earlier setup steps are done; the model must continue from where it stopped, not repeat finished work.

Scoring: the model picks among labeled candidates for five fields (`goal`, `file`, `constraint`, `detail`, `next_step`) via exact match. All compact artifacts share the same 85-word budget, frozen before evaluation.

Models: `qwen3.5:4b` (primary) and `qwen3.5:9b` (robustness), via Ollama, temperature 0.

4. What a State Sheet Looks Like

A concrete example from our dataset — the state sheet stores five explicit fields vs. a bullet summary's freeform narrative:

C3 State Sheet (structured)

```
file: webhook_consumer.py
goal: add retry logic to webhook handler
constraint: max 3 retries, exponential backoff
detail: switched from schema.py after task #4
next_step: implement try/except in process_event()
```

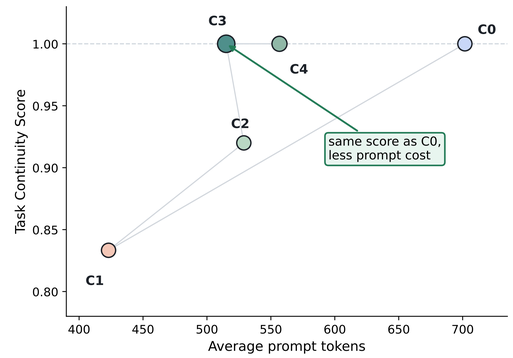
C2 Bullet Summary (freeform)

- We discussed adding retry logic to the webhook handler with exponential backoff.
- The plan is to wrap the processing function in a try/except block.
- Earlier we looked at the schema file but moved on.

× **Missing:** no filename → model edits wrong file

The state sheet's explicit `file:` field prevents the most common error. Bullet summaries mention "the schema file" but never record which file the agent should actually edit next.

5. Main Result: Same Accuracy, Fewer Tokens



−26.6%	−0.52s
prompt tokens vs. full transcript	avg. latency vs. full transcript

The state sheet (C3) matches the full conversation history at 1.000 accuracy while using 26.6% fewer prompt tokens and finishing each call 0.52 s faster.

The hybrid sheet (C4) also scores perfectly but uses more tokens than C3. Adding raw detail on top of the structured fields did not help—organization matters more than volume.

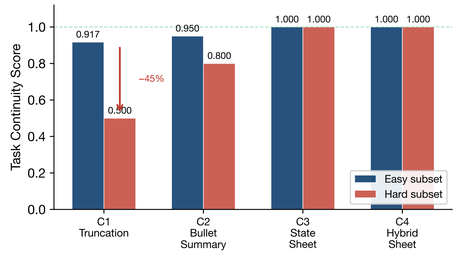
6. Where Bullet Summaries Go Wrong

Bullet summaries (C2) get the goal and next step right—but pick the wrong file in all 9 of their errors. The freeform notes capture the "story" but drop the specific filename.

Truncation (C1) fails more broadly: mixed state (4), lost constraints (4), wrong files (3), and lost details (2).

State sheets (C3/C4) store each fact in a dedicated field. Result: zero errors.

7. Robustness: Bigger Model, Same Ranking



Model	C1	C2	C3	C4
`qwen3.5:4b`	0.833	0.920	**1.000**	**1.000**
`qwen3.5:9b`	0.920	0.960	**1.000**	**1.000**

Scaling from 4B to 9B helps the weaker formats (C1 and C2 both improve) but does not change the ranking. The state sheet wins at both model sizes.

The hard/easy chart tells the real story: on easy tasks, even truncation scores 0.917. On hard tasks—where recent messages are misleading—it crashes to 0.500. The memory format only matters when the context is adversarial.

8. Takeaways

- Dropping old messages loses critical details, especially when recent turns are misleading.
- Freeform summaries capture the overall story but forget which file to edit—a single missing anchor derails the whole action.
- A five-field state sheet fixes this by giving each key fact its own slot. It matches the full transcript at 26.6% fewer tokens.
- Adding more raw detail (C4) does not help—structure beats volume.

Bottom line: if your coding agent needs a compaction policy, start with a short structured schema, not a longer freeform summary.

Limitations & Future Work

- 30 tasks: clear trends, but too few for significance tests.
- Closed-choice scoring inflates C3/C4 by simplifying the task.
- We only tested one model family (`qwen3.5`); other architectures may behave differently.
- We used a single word budget (85 words); scaling behavior is unexplored.

Related Work

LoCoMo [1] benchmarks multi-session recall; Jia et al. [2] show recall degrades unevenly across information types.

Chuang et al. [3] learn natural-language compression; EcoRAG [4] compresses retrieval context for RAG.

HiAgent [5] and Memory-Augmented Coding Agents [6] build hierarchical memory layers. Our work is simpler: we compare frozen text artifacts, not learned systems.

References

[1] Maharana et al. (2024). *LoCoMo.* ACL. [2] Jia et al. (2025). *Evaluating Memory Quality.* NAACL. [3] Chuang et al. (2024). *Learning Prompt Compression.* EMNLP. [4] Jeong et al. (2025). *EcoRAG.* ICLR. [5] Hu et al. (2025). *HiAgent.* NAACL. [6] Kang et al. (2025). *Memory-Augmented Coding Agents.* ICLR.

Code & Data: `https://github.com/NumerousJLs/context-compaction-paper`